Remote Sensing Foundation Models

This notebook demonstrates how to discover, filter, and load remote sensing foundation models using the `geoai` package. The catalog is drawn from the Awesome Remote Sensing Foundation Models curated list and integrates with TerraTorch for model loading and fine-tuning.

What you will learn: - Browse the full catalog of remote sensing foundation models - Filter by category, modality, task, or framework support - Retrieve detailed metadata for any model - Load a model backbone via TerraTorch for inference or fine-tuning

Install Package

Run the cell below to install `geoai-py` and `terratorch`. Both are needed if you want to load model weights; catalog and filtering work without `terratorch`.

```
%pip install -q "geoai-py[terratorch]"
```

Import Libraries

```
import geoai
```

List All Foundation Models

Call `geoai.list_foundation_models()` with no arguments to see the full catalog as a pandas DataFrame. Each row is one registered model with summary metadata.

```
df = geoai.list_foundation_models(verbose=False)
df
```

Filter by Category

Models are organised into five categories:

Category	Description
`vision`	Self-supervised vision-only models (MAE, contrastive, etc.)
`vision-language`	Models combining image and text understanding (VLM, SAM, etc.)
`generative`	Diffusion and generative models
`vision-location`	Geography- and location-aware models
`agents`	Agent-based earth observation systems

```
df_vision = geoai.list_foundation_models(category="vision", verbose=False)
df_vision
```

```
df_vlm = geoai.list_foundation_models(category="vision-language",
verbose=False)
df_vlm
```

Filter by Modality

Filter to a specific sensor modality such as `"multispectral"`, `"sar"`, `"hyperspectral"`, `"multimodal"`, `"timeseries"`, `"optical"`, or `"lidar"`.

```
df_ms = geoai.list_foundation_models(modality="multispectral", verbose=False)
df_ms
```

```
df_hyper = geoai.list_foundation_models(modality="hyperspectral",
verbose=False)
df_hyper
```

Filter by Downstream Task

Use `task=` to keep only models that list a particular downstream capability.

```
df_seg = geoai.list_foundation_models(task="segmentation", verbose=False)
df_seg
```

```
df_reg = geoai.list_foundation_models(task="regression", verbose=False)
df_reg
```

TerraTorch-Supported Models

Set `terratorch_only=True` to see only models you can load directly via `geoai.load_foundation_model()` without custom code.

```
df_tt = geoai.list_foundation_models(terratorch_only=True, verbose=False)
df_tt
```

Models Available on HuggingFace Hub

Set `huggingface_only=True` to see only models with a HuggingFace identifier, making it easy to load them with `transformers` or `huggingface_hub`.

```
df_hf = geoai.list_foundation_models(huggingface_only=True, verbose=False)
df_hf
```

Combine Filters

Filters compose — apply category, modality, task, and framework flags together.

```
df_combined = geoai.list_foundation_models(
    category="vision",
    modality="multispectral",
    task="segmentation",
    terratorch_only=True,
    verbose=False,
)
df_combined
```

Get Detailed Model Info

`geoai.get_foundation_model_info()` returns the full metadata dictionary for a single model, including paper URL, code URL, license, and description.

```
info = geoai.get_foundation_model_info("prithvi-eo-2.0-300m")
for key, value in info.items():
    print(f"{key}: {value}")
```

```
info_clay = geoai.get_foundation_model_info("clay-v1")
for key, value in info_clay.items():
    print(f"{key}: {value}")
```

Check TerraTorch Availability

Before loading a model, verify that TerraTorch is installed.

```
print(f"TerraTorch available: {geoai.check_terratorch_available()}")
```

Load a Model via TerraTorch

`geoai.load_foundation_model()` uses TerraTorch's backbone registry to download and instantiate the model. A GPU is not required for the registry call itself, but inference will be faster with one.

> **Note:** TerraTorch must be installed (`pip install "geoai-py[terratorch]"`) and the model must be listed as `terratorch_supported: True` in the catalog.

TerraTorch Backbone Keys

Each TerraTorch-supported model has an exact backbone key used internally by `geoai.load_foundation_model()`. The table below shows the mapping.

```
df_keys = geoai.list_foundation_models(terratorch_only=True, verbose=False)[
    ["name", "abbreviation", "modality", "publication", "terratorch_key"]
]
df_keys
```

```python
try:
    model = geoai.load_foundation_model("prithvi-eo-2.0-300m",
pretrained=True)
    n_params = sum(p.numel() for p in model.parameters())
    print(f"Model type : {type(model).__name__}")
    print(f"Parameters : {n_params:,}")
    print(f"Trainable  : {sum(p.numel() for p in model.parameters() if
p.requires_grad):,}")
    print()
    print("Model loaded successfully — ready for fine-tuning or inference.")
except ImportError as e:
    print(f"[!] TerraTorch not installed: {e}")
    print("    Fix: pip install "geoai-py[terratorch]"")
except NotImplementedError as e:
    print(f"[!] Model not yet TerraTorch-supported:\n    {e}")
except RuntimeError as e:
    print(f"[!] TerraTorch registry error:\n    {e}")
```